

Monorepo Structure

Project Name

CarbonWise

Version: 1.0

Status: Approved

Repository Type: Monorepo

1. Purpose

This document defines the repository organization, project structure, ownership boundaries, shared libraries, development workflow, and code organization strategy for CarbonWise.

The repository structure is designed to maximize:

- Code Quality (High Impact)
- Maintainability
- Scalability
- Testability
- Security

while remaining simple enough for challenge evaluation.

2. Repository Principles

The repository follows:

Separation of Concerns

Business logic should not mix with UI logic.

Modular Development

Features should be independently maintainable.

Reusability

Shared code belongs in dedicated packages.

Testability

Every major module must support unit and integration testing.

3. Repository Structure

```text id="repo-root" carbonwise/

```
|—— apps/ | |—— packages/ | |—— infrastructure/ | |—— tests/ | |—— docs/ | |—— scripts/
| |—— .github/ | |—— .env.example | |—— package.json | |—— README.md
```

---

### # 4. Apps Directory

Contains deployable applications.

```text id="apps-root"

apps/

```
|— web/
|
└— api/
```

5. Frontend Application

Location:

```text id="frontend-path" apps/web

Technology:

```text id="frontend-stack"

Next.js 15

TypeScript

Tailwind CSS

Shadcn/UI

Structure

```text id="frontend-structure" web/

|— src/ | |— public/ | |— tests/ | └— package.json

---

# Source Structure

```text id="frontend-src"

src/

|— app/

|— components/

|— features/

|— hooks/

|— services/

|— store/

|— types/

|— utils/

└— lib/

App Directory

Contains:

```text id="app-dir" Pages

Layouts

Routing

Metadata

```

Components Directory

Contains reusable UI.

Examples:

```text id="components-examples"
Button

Card

Modal

Input

Chart
```

Features Directory

Contains feature-specific modules.

```text id="features-list" features/

```
|—— auth/ | |—— dashboard/ | |—— calculator/ | |—— recommendations/ | |—— goals/ |
|—— carbon-twin/
```

```

Why Feature-Based?

Supports:
```

```
```text id="quality-impact"
Code Quality (High Impact)
```

by reducing coupling.

6. Backend Application

Location:

```
```text id="backend-path" apps/api
```

Technology:

```
```text id="backend-stack"
NestJS

TypeScript
```

Structure

```
```text id="backend-structure" api/
```

```
|—— src/ | |—— test/ | |—— package.json
```

```

Backend Source Structure

```text id="backend-src"
src/

|—— modules/
|
|—— common/
|
|—— config/
|
|—— database/
```

```
|  
└─ main.ts
```

Modules Directory

Contains business domains.

```
```text id="modules-list" modules/
```

```
├── auth/ | ├── users/ | ├── activities/ | ├── carbon/ | ├── recommendations/ | ├──
goals/ | ├── analytics/ | ├── carbon-twin/ | └── notifications/
```

```

Example Module Structure

```text id="module-example"  
auth/  
  
├─ controllers/  
│  
├─ services/  
│  
├─ dto/  
│  
├─ entities/  
│  
├─ guards/  
└─ tests/
```

Why This Structure?

Supports:

- SOLID Principles
- Maintainability
- Testability

7. Shared Packages

Location:

```
```text id="shared-path" packages/
```

```

Structure

```text id="packages-structure"  
packages/  
  
├─ ui/  
│  
├─ validation/  
│  
├─ types/  
│  
├─ constants/  
│  
└─ utilities/
```

UI Package

Contains:

```
```text id="ui-package" Buttons
```

Cards

Forms

Charts

Used by:

```
```text id="ui-consumers"  
Frontend
```

Validation Package

Contains:

```text id="validation-package" Zod Schemas

Input Validators

Used by:

```text id="validation-consumers"  
Frontend

Backend

Types Package

Contains:

```text id="types-package" Shared Interfaces

API Contracts

Enums

---

# Utilities Package

Contains:

```text id="utilities-package"  
Formatters

Date Helpers

Calculation Helpers

8. Database Layer

Location:

```text id="database-path" apps/api/src/database

```

Structure

```text id="database-structure"
database/

├─ prisma/
│
├─ migrations/
│
└─ seed/
```

Prisma Directory

Contains:

```text id="prisma-files" schema.prisma

generated/

```

Migrations Directory

Contains:

```text id="migration-files"
Database Migrations
```

Seed Directory

Contains:

```text id="seed-files" Development Data

Emission Factors

```

9. Infrastructure Directory

Location:

```text id="infra-path"  
infrastructure/
```

Structure

```text id="infra-structure" infrastructure/

|—— docker/ | |—— nginx/ | |—— monitoring/ | |—— deployment/

```

Docker

Contains:

```text id="docker-files"  
Dockerfiles  
  
docker-compose.yml
```

Monitoring

Contains:

```text id="monitoring-files" Metrics

Alerts

Dashboards

```

```

```
Deployment
```

```
Contains:
```

```
```text id="deployment-files"
```

```
Deployment Scripts
```

```
Release Configurations
```

10. Testing Structure

Location:

```
```text id="testing-path" tests/
```

```

```

```
Structure
```

```
```text id="testing-structure"
```

```
tests/
```

```
|— unit/
```

```
|
```

```
|— integration/
```

```
|
```

```
|— e2e/
```

```
|
```

```
└— accessibility/
```

Unit Tests

Technology:

```
```text id="unit-tech" Jest
```

```

Integration Tests

Technology:

```text id="integration-tech"
Supertest
```

End-to-End Tests

Technology:

```
```text id="e2e-tech" Playwright
```

```

Accessibility Tests

Technology:

```text id="accessibility-tech"
axe-core

Playwright
```

11. Documentation Structure

Location:

```
```text id="docs-path" docs/
```

```

Structure

```text id="docs-structure"
docs/
```

```
|— architecture/
|
|— api/
|
|— database/
|
|— testing/
|
|— deployment/
```

Purpose

Maintain project transparency.

Supports:

```text id="maintainability" Maintainability

```

12. GitHub Structure

Location:

```text id="github-path"
.github/
```

Structure

```text id="github-structure" .github/

```
|— workflows/ | — templates/
```

```

Workflows

Examples:

```text id="workflow-list"
```

```
ci.yml  
security.yml  
deploy.yml
```

13. Security Organization

Supports:

```text id="security-impact" Security (Medium Impact)

```

Sensitive Data

Never stored in:

```text id="forbidden-storage"  
Source Code  
  
Repository  
  
Commits
```

Stored in:

```text id="allowed-storage" Environment Variables

```

Security Components

```text id="security-components"  
JWT  
  
Helmet  
  
bcrypt  
  
Rate Limiter
```

14. Performance Organization

Supports:

```text id="efficiency-impact" Efficiency (Medium Impact)

```

Redis Layer

Location:

```text id="redis-location"  
apps/api/src/modules/cache
```

Cached Resources

```text id="cached-resources" Dashboard

Recommendations

Analytics

Carbon Twin Results

```

Database Optimization

Includes:

```text id="db-optimization"  
Indexes  
  
Pagination
```

15. Code Standards

Supports:

```text id="code-quality-impact" Code Quality (High Impact)

```

Linting

Tool:

```text id="eslint-tool"  
ESLint
```

Formatting

Tool:

```text id="prettier-tool" Prettier

```

Naming Conventions

Files

```text id="file-naming"  
kebab-case
```

Example:

```text id="file-example" carbon-score.service.ts

```

Classes

```text id="class-naming"
PascalCase
```

Example:

```
```text id="class-example" CarbonCalculatorService
```

```

Variables

```text id="variable-naming"
camelCase
```

Example:

```
```text id="variable-example" carbonScore
```

```

16. Accessibility Organization

Supports:

```text id="accessibility-impact"
Accessibility (Low Impact)
```

Accessibility validation exists inside:

```
```text id="accessibility-folder" tests/accessibility
```

```

Checks include:

- Keyboard Navigation
```

- Color Contrast
- Screen Reader Support
- ARIA Compliance

---

## # 17. Repository Compliance

Challenge Requirements:

- ✓ Public Repository
- ✓ Single Branch
- ✓ Complete Source Code
- ✓ README Included
- ✓ Under 10 MB

---

## # 18. Future Expansion

Architecture supports future modules:

```text id="future-modules"  
AI Carbon Coach

Community Challenges

Smart Device Integrations

Enterprise Dashboard

Advanced Forecasting

without restructuring.

19. Success Criteria

The repository structure is successful when:

- ✓ Code is easy to navigate

- ✓ Features remain modular
- ✓ Tests are organized
- ✓ Security concerns are isolated
- ✓ Shared code is reusable
- ✓ Development remains scalable
- ✓ New contributors can onboard quickly
- ✓ Code Quality remains high throughout the project lifecycle